

Labwork 3

Truong Quang Nam
Student ID: 2540080

1 Device

- **GPU** NVIDIA GeForce GTX 1650
- **CPU** Ryzen 7 4800h

Image to process



Figure 1: A image to process

2 Cpu compute

This part to convert the normal image "Image.jpg" to gray image by cpu
Step to do:

```
img = mpimg.imread('Image.jpg')
height, width, channels = img.shape
pixel_count = img.shape[0] * img.shape[1]
```

Load image and compute size and number pixel of image

```
img_flattened = img.reshape(-1, 3)
gray_flattened = np.zeros(pixel_count, dtype=np.float32)
```

Flat the image in 1D and create a gray image to save the output

```
start_time = time.time()
for index in range(pixel_count):
    r = img_flattened[index,0] / 3.0
    g = img_flattened[index,1] / 3.0
    b = img_flattened[index,2] / 3.0
    gray_flattened[index] = r + g + b
end_time = time.time()
```

Start time counter and compute avagerate r g b and put to save to gray image with each index

```
gray_img = gray_flattened.reshape(height, width).astype(np.uint8)
plt.figure(figsize=(6, 6))
plt.title("Gray image ")
plt.imshow(gray_img, cmap='gray')
plt.axis('off')
plt.imsave('gray_iamge_cpu.png', gray_img, cmap='gray')
plt.show()
```

Display and save the image

3 GPU Compute

This part to convert the normal image "Image.jpg" to gray image by gpu
Step to do:

```
@cuda.jit
def gray_image(image_flat, gray_flat, pixel_count):

    index = cuda.grid(1)

    if index < pixel_count:

        r = image_flat[index, 0]
        g = image_flat[index, 1]
        b = image_flat[index, 2]

        gray_flat[index] = (r + g + b) / 3.0
```

Here is function to compute in gpu. Read image as cpu

```
d_img_flattened = cuda.to_device(img_flattened)
d_gray_flattened = cuda.device_array(pixel_count, dtype=np.float32)
```

Load image to gpu memory, create memory in gpu to save output in

```
threads_per_block = 256
blocks_per_grid = (pixel_count + (threads_per_block - 1))

gray_image[blocks_per_grid, threads_per_block](
    d_img_flattened,
    d_gray_flattened,
    pixel_count
)

cuda.synchronize()

gray_flattened_cpu = d_gray_flattened.copy_to_host()
```

Define thread to process and call function to run, synchronize wait for all done, then copy from gpu memory to host memory The part to display and save are the same as cpu

4 Result

CPU time: 86ms GPU Time: 463ms



Figure 2: CPU process



Figure 3: GPU process